

1. Abstract

This project presents the design and implementation of a two-player Pong game using a Complex Programmable Logic Device (CPLD). The game is displayed on an 8x8 LED dot matrix, where players control paddles to interact with a bouncing ball, aiming to score by preventing the ball from passing their paddle. The design utilizes multiplexing to manage the display of the paddles and ball, while a 1 kHz clock frequency ensures smooth transitions between the display states. The game logic incorporates paddle control, ball movement, collision detection, and scoring. Input buttons are used to move the paddles, and the game updates every 200 ms to maintain fluid gameplay. A seven-segment display is used to show the score, which halts when one player reaches 9 points. The project demonstrates the application of digital design principles, such as multiplexing, state machines, and real-time control, to create a fully functional interactive game using CPLD hardware. The result is an efficient, hardware-based Pong game with real-time processing and simple controls, offering insight into the use of programmable logic for game development.

2. Introduction

This report details the design and implementation of a two-player Pong game using a Complex Programmable Logic Device (CPLD). Pong is a classic arcade game where two players control paddles on either side of the screen, with the goal of preventing a ball from passing their paddle while trying to score by sending the ball past their opponent's paddle. In this project, the game is displayed using an 8x8 LED dot matrix, which visually represents the paddles and the ball. To manage the display, a multiplexing technique is employed to switch between displaying the paddles and the ball, making sure all elements appear visible simultaneously. The game mechanics include paddle control, ball movement, and scoring logic, each implemented in hardware using the CPLD.

The system's core is driven by a 10 MHz clock, which is divided down to a 1 kHz signal for multiplexing the dot matrix display. Players control their paddles using input buttons, and the game logic ensures that the paddles stay within the bounds of the matrix. The ball's movement is tracked and updated based on its speed and direction, with collision detection ensuring realistic ball behavior when it bounces off the paddles or walls. If the ball reaches the top or bottom row without being hit, a point is scored for the opposing player, and the ball is reset to the center.

The game is designed to stop once a player scores 9 points, with the final score displayed on a seven-segment display. This project demonstrates how hardware can be used to create an interactive game, highlighting the application of digital design techniques such as multiplexing, state machine implementation, and collision detection, all within the context of a simple yet engaging gaming experience. By utilizing the CPLD, the project also showcases the potential of programmable logic in developing real-time interactive systems with efficient resource utilization.

3. Design

3.1 Problem Formulation

3.1.1 Identification of Scope

The **Two Player Pong Game** is a simplified version of the classic arcade game, implemented using **Verilog HDL** on a **CPLD (Complex Programmable Logic Device)**. The game runs on an **8x8 LED matrix display**, where two players control paddles to hit a moving ball back and forth. The paddles are represented by **green LEDs** on either side of the matrix, while the ball is displayed using a **red LED**.

The scope of the project includes the following objectives:

- Designing and implementing the game logic using Verilog.
- Ensuring smooth ball movement and accurate paddle control.
- Controlling the **8x8 LED matrix** efficiently using multiplexing techniques.
- Integrating input switches for two players to control the paddle positions.
- Simulating the design to validate functionality and implementing it on a CPLD for testing.
- Providing user-friendly controls and achieving minimal response time.

The project showcases a combination of **digital circuit design**, **hardware interfacing**, and **interactive game mechanics** using programmable logic.

3.1.2 Literature Review

To successfully design and implement the Two Player Pong Game, research was conducted in the following areas:

1. **LED Matrix Interfacing:**
The **8x8 LED matrix** is used to display game elements. The matrix works using **row and column multiplexing**, where specific rows and columns are activated to light up desired LEDs. This technique reduces the number of control pins required.
2. **Verilog HDL (Hardware Description Language):**
Verilog is used to implement the game logic, including paddle movement, ball motion, and collision detection. Digital circuits are designed using combinational and sequential logic.
3. **CPLD Architecture:**
CPLDs are ideal for small to medium digital systems due to their low cost, reprogrammability, and deterministic performance. They provide sufficient logic resources to implement the game efficiently.
4. **Classic Pong Game Mechanics:**
The Pong game features two paddles and a ball. The ball moves continuously across the screen, bouncing off paddles and boundaries. Players control their paddles to prevent the ball

from crossing their side. The simplicity of this game makes it ideal for CPLD implementation.

This research enabled the development of a clear approach to solving the problem, focusing on hardware control, game logic, and player interactivity.

3.1.3 Formulation of Problem

The problem requires implementing an interactive **Two Player Pong Game** with the following constraints and challenges:

1. **Game Logic Implementation:**
 - Accurate control of ball movement across the LED matrix.
 - Detection of collisions with paddles and boundaries.
 - Smooth and responsive movement of player-controlled paddles.
2. **LED Matrix Control:**

The 8x8 LED matrix must be controlled efficiently using multiplexing, lighting up the correct positions for the ball and paddles in real time.
3. **Player Input Integration:**

Two players must be able to control their paddles using input switches for **UP** and **DOWN** movements.
4. **Real-Time Updates:**

The design must ensure minimal delays to maintain smooth ball motion and paddle responsiveness.

The formulated problem is: “**How to design a two-player interactive game on CPLD that controls an 8x8 LED matrix and implements the game mechanics of Pong using Verilog?**”

3.1.4 Analysis

The analysis of the project divides the design into components:

1. **Game Components:**
 - **Ball:** A red LED that moves in both X and Y directions. Its position is updated at regular intervals.
 - **Paddles:** Green LEDs on the left and right sides of the matrix, controlled by players.
2. **Game Logic:**
 - **Ball Movement:** The ball's position is updated based on its direction and speed.
 - **Collision Detection:** If the ball hits a paddle, it bounces back. If it crosses a player's side, the game can end or reset.
 - **Paddle Control:** Paddle positions are updated based on player input switches.

3. LED Matrix Control:

The **8x8 LED matrix** is addressed using row-column multiplexing. For each cycle, the row and column drivers activate the LEDs corresponding to the ball and paddle positions.

4. Inputs:

Two sets of switches (UP/DOWN) are used to control paddle movement.

This analysis ensures a modular approach, breaking the project into manageable parts that can be implemented step by step.

3.2 Design Method (PO(a))

The project is designed using the following step-by-step methodology:

1. Game Logic Design:

- A **Finite State Machine (FSM)** is used to control the game flow. States include ball movement, paddle control, and collision handling.

2. LED Matrix Implementation:

- Row and column multiplexing are used to control the LEDs efficiently.

3. Player Input Handling:

- Two sets of switches are mapped to control paddle movement.

4. Simulation:

- The Verilog code is simulated using software tools Quartus 8.1 to verify ball motion, paddle control, and collision detection.

5. Hardware Implementation:

- The design is synthesized and programmed onto a CPLD using development tools.

6. Testing and Debugging:

- The final hardware implementation is tested for smooth game execution and accuracy.
-

3.3 Hardware Design

The hardware components of the project include:

1. **CPLD Board:** Executes the Verilog game logic.
2. **8x8 LED Matrix:** Displays the ball and paddle positions dynamically.
3. **Input Switches:** Two sets of UP/DOWN switches are connected to control paddle movement.
4. **Power Supply:** Provides power to the CPLD and LED matrix.

The CPLD processes the game logic and updates the LED matrix based on player input and game events.

3.4 Full Source Code of Firmware

```
module paddle_test(
    input clk,
    input reset,
    input up_p1, down_p1,
    input up_p2, down_p2,
    output reg [7:0] rows,
    output reg [7:0] colsr,
        output reg [7:0] colsg,
        output [7:0] segments,
        output [2:0] cathode_sel
);

    reg [2:0] p1_pos, p2_pos;          // Paddle positions
    reg [2:0] ball_x, ball_y;          // Ball position
    reg [1:0] ball_speed_x, ball_speed_y; // Ball speed direction
        reg [3:0] score_p1, score_p2; // Player scores
    reg [1:0] display_state;           //counter for multiplexing
        wire counter_0;

    // Clock divider for generating slower display and update clocks
    wire clk_1k, clk_10;
        clock_divider clk_gen1(clk, clk_1k); // 1 kHz clock for display
multiplexing
        clock_divider_10 clk_gen2(clk, clk_10, counter_0); // 10 Hz clock for game
logic updates

    // Initialization block
    initial
        begin
            rows = 8'b00000000;
```

```

        colsr = 8'b00000000; // red led
        colsg = 8'b00000000; // green led

        p1_pos = 2;           // Initial position for paddle 1
        p2_pos = 2;           // Initial position for paddle 2
        ball_x = 4;           // Initial position for the ball
        ball_y = 4;

        ball_speed_x = 0;
        ball_speed_y = 1;

        score_p1 = 0;
        score_p2 = 0;
        display_state <= 2'b00; // Start with displaying paddle 1
    end

    // Display Score
    seven_seg_test display(clk, reset, score_p1, score_p2, segments,
cathode_sel);

    // Sequential logic for updating paddle positions and ball mechanics
    always @(posedge clk_10 or posedge reset)
        begin
            if (reset)
                begin
                    p1_pos = 2;
                    p2_pos = 2;
                    ball_x = 4;
                    ball_y = 4;
                    ball_speed_x = 0;

```

```

        if(counter_0)
            ball_speed_y = 1;
        else
            ball_speed_y = 2;

        score_p1 = 0;
        score_p2 = 0;
    end
    else
        begin
            // Paddle 1 controls
            if (~up_p1 && p1_pos > 0) // Move paddle 1
                p1_pos = p1_pos - 1;
            if (down_p1 && p1_pos < 5) // Move paddle 1
                p1_pos = p1_pos + 1;

            // Paddle 2 controls
            if (~up_p2 && p2_pos > 0) // Move paddle 2
                p2_pos = p2_pos - 1;
            if (down_p2 && p2_pos < 5) // Move paddle 2
                p2_pos = p2_pos + 1;

            // Ball mechanics
            if (ball_y == 0)
                begin
                    if (ball_x >= p1_pos && ball_x
<= p1_pos + 2) begin
                        if(ball_x ==p1_pos)

```

Reverse direction

1; // Player 2 scores

=0;

=0;

```
        ball_speed_x = 2;
    else if(ball_x ==p1_pos+1)
        ball_speed_x = 0;
    else if(ball_x ==p1_pos+2)
        ball_speed_x = 1;

        ball_speed_y = 1; //

    end
    else
        begin

            score_p2 = score_p2 +

            if(score_p2>=9) begin
                ball_speed_x

                ball_speed_y

                ball_x = 4;
                ball_y = 4;
            end
            else begin
                p1_pos = 2;
                p2_pos = 2;
                ball_x = 4;
                ball_y = 3;

                ball_speed_x =0;
                ball_speed_y = 1;
            end
        end
    end
```

```

end
else if (ball_y== 7)
begin
    if (ball_x >= p2_pos && ball_x
    <= p2_pos + 2) begin
        case(ball_x)
            p2_pos:
                ball_speed_x = 2;
            p2_pos+1:
                ball_speed_x = 0;
            p2_pos+2:
                ball_speed_x = 1;
        endcase
        Reverse direction
        ball_speed_y = 2; //
    end
else
begin
    score_p1 = score_p1 + 1; //
    Player 2 scores

    if(score_p1>=9) begin
        ball_speed_x =0;
        ball_speed_y =0;
        ball_x = 4;
        ball_y = 4;
    end
    else begin
        p1_pos = 2;
        p2_pos = 2;
        ball_x = 4;

```

```

        ball_y = 5;

        ball_speed_x = 0;
        ball_speed_y = 2;
    end
end

end

else begin

    // Ball boundary checks
    if (ball_x == 7)
        ball_speed_x = 2;
    else if (ball_x == 0)
        ball_speed_x = 1;
    end

    // Update ball position
    if (ball_speed_x == 1 && ball_x < 7)
        ball_x = ball_x + 1;
    else if (ball_speed_x == 2 && ball_x > 0)
        ball_x = ball_x - 1;
    else if (ball_speed_x == 0)
        ball_x = ball_x;

    if (ball_speed_y == 1 && ball_y < 7)
        ball_y = ball_y + 1;
    else if (ball_speed_y == 2 && ball_y > 0)
        ball_y = ball_y - 1;

end
end

```

```

        end

//counter for multiplexing display logic
always @(posedge clk or posedge reset) begin
    if (reset) begin
        display_state = 2'b00; // Reset display state
    end else begin
        display_state = display_state + 1; // Cycle through states
    end

    case (display_state)
        2'b00: begin // Display paddle 1
            rows = 8'b00000000; // Clear rows
            colsg = 8'b00000000;

            rows[p1_pos] = 1;
            rows[p1_pos + 1] = 1;
            rows[p1_pos + 2] = 1;
            colsg[0] = 1;
        end
        2'b01: begin // Display paddle 2
            rows = 8'b00000000; // Clear rows
            colsg = 8'b00000000; // Active-low for paddle 2
            rows[p2_pos] = 1;
            rows[p2_pos + 1] = 1;
            rows[p2_pos + 2] = 1;
            colsg[7] = 1;
        end
    end
end

```

```

        2'b10: begin // Display ball
            rows = 8'b00000000;    // Clear rows
            colsr = 8'b00000000;
            colsg = 8'b00000000;

            rows[ball_x] = 1;
            colsr[ball_y] = 1;
            colsg[ball_y] = 1;
        end
        default: begin
            rows = 8'b00000000;
            colsr = 8'b00000000;
            colsg = 8'b00000000;
        end
    endcase
end

endmodule

```

4. Implementation

The game was implemented in three main parts:

1. Dot Matrix Control
2. Game Mechanics
3. Seven-Segment Control

4.1 Dot Matrix Control

An 8x8 LED dot matrix was used to display the game, as shown in Figure 2. The dot matrix consists of 64 red and green LEDs arranged in a grid. Each LED is controlled by its row and column, where the cathodes are connected to rows and the anodes to columns. This configuration enables individual control of each LED.

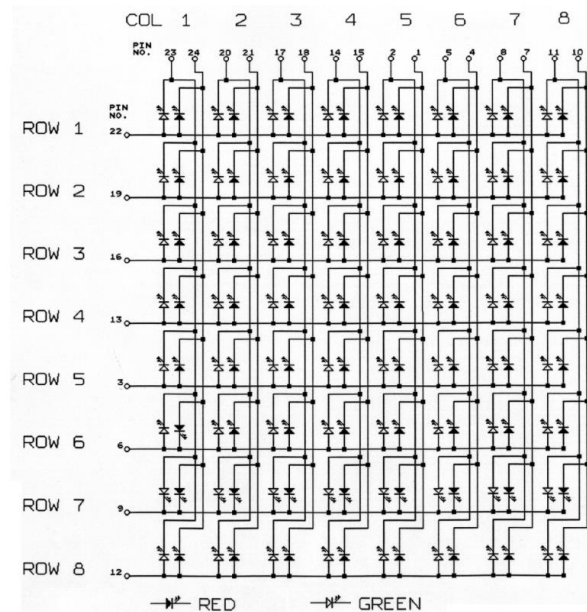


Figure 2: (Left) LED Dot Matrix (Right) Schematics of Dot Matrix

To independently display the paddles and the ball, multiplexing was employed. The system uses a 10 MHz clock divided down to 1 kHz to switch between displaying paddle 1, paddle 2, and the ball at a high enough frequency that all appear visible simultaneously. A 2-bit variable, `display_state`, cycles through the three display states:

1. Paddle 1
2. Paddle 2
3. Ball

Pseudo-code for Dot Matrix Control:

```
IF clock_1kHz_edge THEN
  SWITCH display_state:
    CASE 0: // Display Paddle 1
      Clear all rows and columns
      Activate rows corresponding to paddle 1 position
      Enable column 0 for paddle 1
    CASE 1: // Display Paddle 2
      Clear all rows and columns
      Activate rows corresponding to paddle 2 position
      Enable column 7 for paddle 2
    CASE 2: // Display Ball
      Clear all rows and columns
      Activate row corresponding to ball_x
      Enable column corresponding to ball_y
    DEFAULT:
```

```
        Clear all rows and columns
    END SWITCH
END IF
```

After multiplexing the game display looks like the following,

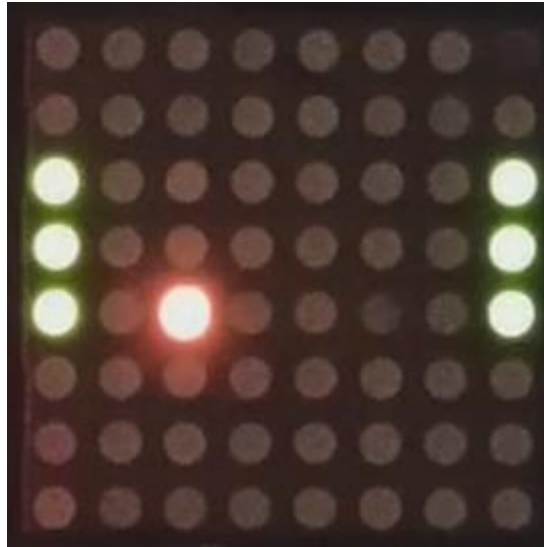


Figure 3: Game Display

Here green leds are used to represent the paddles and both red and green leds are used to represent the ball.

4.2 Game Mechanics

The core game logic includes three components: Paddle Control, Ball Movement, and Scoring Logic. The game operates at a reduced update frequency of 5 Hz (from the 10 Hz clock) to ensure smooth gameplay.

4.2.1 Paddle Control

Each paddle is controlled by two inputs: up and down. Paddle 1's position is stored in the p1_pos register, and Paddle 2's position is stored in p2_pos. Both positions are constrained between 0 and 5 to prevent the paddles from moving outside the matrix.

Pseudo-code for Paddle Control:

```
IF up_p1_pressed AND p1_pos > 0 THEN
    Decrease paddle 1 position
IF down_p1_pressed AND p1_pos < 5 THEN
    Increase paddle 1 position
```

```

IF up_p2_pressed AND p2_pos > 0 THEN
    Decrease paddle 2 position
IF down_p2_pressed AND p2_pos < 5 THEN
    Increase paddle 2 position

```

Pulse switches from the CPLD board are used to control the paddle movement.

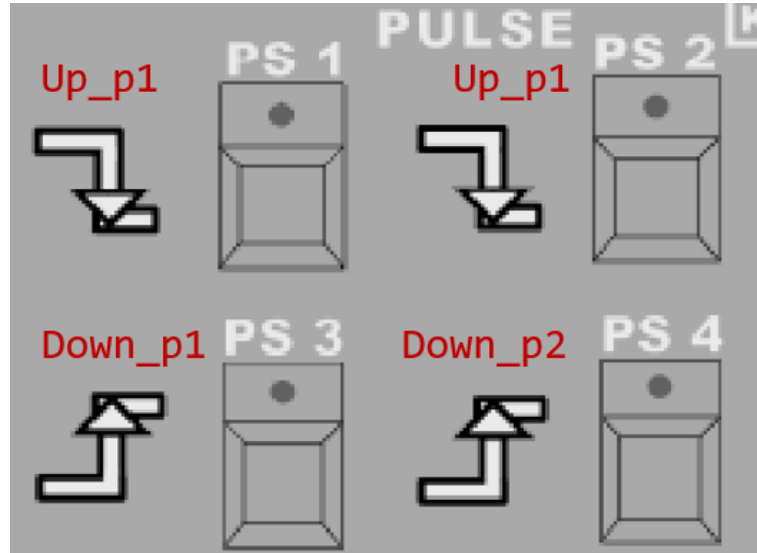


Figure 4: Input Buttons for Paddle Movement

4.2.2 Ball Movement

The ball's position is tracked by `ball_x` and `ball_y`. Its direction and speed are determined by `ball_speed_x` and `ball_speed_y`, respectively. The ball bounces off the paddles and walls. If it reaches the top or bottom rows without being hit by a paddle, a point is scored by the opposing player, and the ball is reset to the center.

Pseudo-code for Ball Movement and Collision:

```

IF ball_y == 0 THEN
    IF ball_x within paddle 1 range THEN
        Adjust ball_x_speed based on collision point
        Reverse ball_y_speed
    ELSE
        Increment player 2 score
        Reset ball to center
    END IF
ELSE IF ball_y == 7 THEN
    IF ball_x within paddle 2 range THEN
        Adjust ball_x_speed based on collision point
        Reverse ball_y_speed
    ELSE

```

```

        Increment player 1 score
        Reset ball to center
    END IF
END IF

IF ball_x == 0 OR ball_x == 7 THEN
    Reverse ball_x_speed
END IF
Move ball by updating ball_x and ball_y based on speeds

```

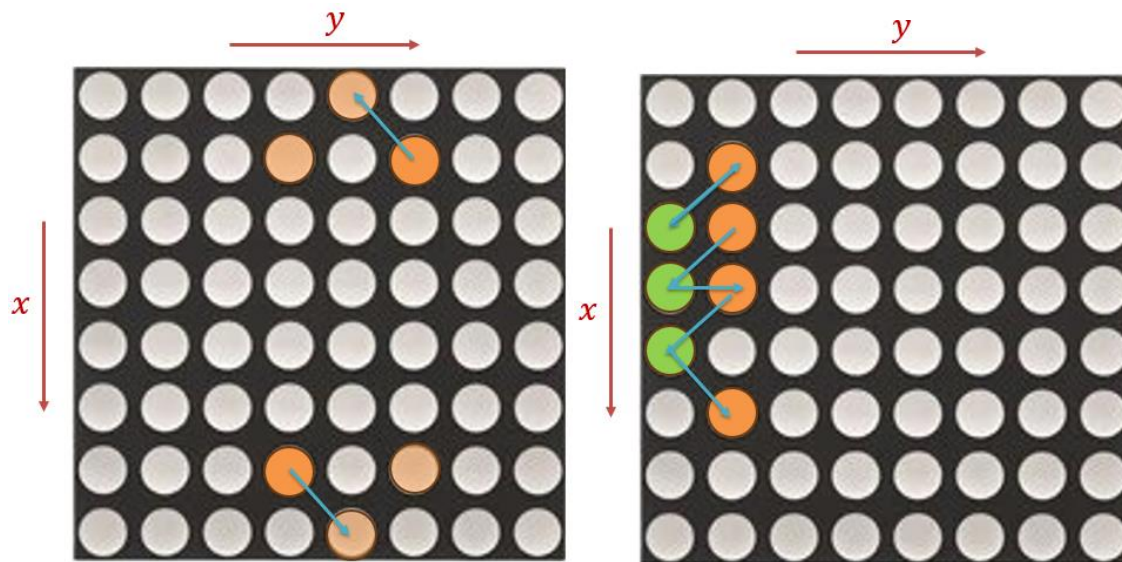


Figure 5: Collisions : (Left) With Boundary (Right) With Paddle

4.2.3 Scoring Logic

Scores for both players are stored in `score_p1` and `score_p2`. When a player scores 9 points, the game stops, and the ball ceases to move. The score display logic is handled by the seven-segment display module, explained in Section 4.3.

Pseudo-code for Scoring Logic:

```

IF score_p1 >= 9 OR score_p2 >= 9 THEN
    Stop ball movement
END IF

```

4.3 Seven-Segment Control

1. Initialize system:

- Set up the clock divider to generate a 1 kHz clock from the system clock.

- Set up the common cathode display control (`cathode_sel`) for two displays: Player 1 and Player 2.
2. On each clock cycle (1 kHz):
 - Check if the reset signal is active:
 - If active, set the current display to Player 1 (`cathode_sel = C1`).
 - If not, toggle the current display between Player 1 and Player 2.
 3. Depending on the current display:
 - If displaying Player 1:
 - Set `cathode_sel` to select Player 1's display (`C1`).
 - Convert Player 1's score to the 7-segment display encoding and output it to segments.
 - If displaying Player 2:
 - Set `cathode_sel` to select Player 2's display (`C6`).
 - Convert Player 2's score to the 7-segment display encoding and output it to segments.
 4. Function to convert score to 7-segment display encoding:
 - Map the input score (0-9) to its corresponding 7-segment encoding.
 - Return the 8-bit segment encoding based on the score.
 5. Repeat the process continuously as the game progresses.

The `seven_seg_test` module is designed to control a two-digit seven-segment display, showing the scores for Player 1 and Player 2 in a Pong game. It takes the system clock (`clk`), a reset signal (`reset`), and the current scores of both players (`score_p1` and `score_p2`) as inputs, and outputs the appropriate signals to drive the display (`segments` and `cathode_sel`).

A clock divider generates a 1 kHz clock from the 10 MHz system clock, used for multiplexing between the two seven-segment displays. The variable `current_display` toggles every 1 kHz clock cycle to alternate between Player 1 and Player 2. When displaying Player 1's score, `cathode_sel` is set to select the first display; when showing Player 2's score, `cathode_sel` selects the sixth display.

The `score_to_segments` function converts the 4-bit score of each player into the corresponding 8-bit value that controls the segments of the seven-segment display. For example, a score of 0 maps to `8'b00111111` to display the number "0," and similarly for other scores from 1 to 9.

In this way, the module alternates between displaying the scores of the two players, refreshing the display at 1 kHz and ensuring smooth updates during the game. The reset signal allows for resetting the game, and the clock divider ensures proper timing for multiplexing and display updates.

5. Design Analysis and Evaluation

5.1 Novelty

The project demonstrates a unique hardware-based implementation of the classic Pong game using a **CPLD**, an 8x8 LED dot matrix, and a seven-segment display. Unlike software-centric systems, this design relies on **pure hardware logic** to control paddle movement, ball dynamics, and scoring logic, making it a creative demonstration of digital circuit design. Multiplexing techniques were utilized to independently display the paddles and ball on the dot matrix, while a reduced clock frequency ensures smooth and playable game mechanics. The project combines efficient use of hardware resources with classic retro aesthetics, delivering a functional and engaging two-player game.

5.2 Design Considerations (PO(c))

5.2.1 Efficient Hardware Utilization

The design effectively maximized the limited hardware resources of the CPLD. By employing **multiplexing** for the dot matrix display, only a single column or row is active at any given time, cycling through the paddles and ball at high speed to create a visible display. This method reduces the number of active LEDs and optimizes power usage while ensuring the paddles and ball appear simultaneously to the human eye. The **10 MHz clock** was divided to generate a 1 kHz frequency for display multiplexing and a further reduced 5 Hz clock for smooth gameplay updates.

Registers like `p1_pos` and `p2_pos` efficiently track paddle positions within a constrained range of 0 to 5, while `ball_x` and `ball_y` manage the ball's real-time position. Resource constraints were addressed by optimizing the game logic and avoiding unnecessary complexity. Features such as paddle movement, ball collisions, and scoring were implemented with minimal logic gates.

5.2.2 Performance Optimization

To ensure responsive gameplay, the system prioritized input latency and clock synchronization. Paddle movement was made smooth and real-time by directly interfacing button inputs with position

registers (`p1_pos` and `p2_pos`). Input buttons for up and down movements allow precise control of paddle positions without noticeable delays. The ball's movement, controlled by `ball_speed_x` and `ball_speed_y`, was updated at a reduced frequency of **5 Hz** to maintain a playable pace.

The collision logic between the ball, paddles, and screen boundaries was carefully designed. Upon collision with a paddle, the ball's horizontal speed is adjusted based on the collision point, enhancing gameplay dynamics. The system also ensures reliable edge detection by reversing the ball's horizontal speed (`ball_x_speed`) when it hits the display's vertical boundaries.

5.2.3 Scalability

The modular nature of the project allows for future scalability without significant architectural changes. Some potential enhancements include:

- **Dynamic Scoring Display:** Expanding the seven-segment display to include animations or visual feedback for scoring events.
- **Sound Effects:** Adding an audio module to produce tones for paddle hits, ball movement, or scoring.
- **AI-Controlled Paddle:** Introducing automated paddle movement using simple logic to enable single-player mode.
- **Increased Difficulty:** Implementing features like multiple balls, variable ball speeds, or reduced paddle sizes to increase challenge levels.

These improvements could enhance user experience while maintaining the core functionality of the design.

5.3 Performance Analysis and Testing

5.3.1 System Validation

The core game components—paddle movement, ball dynamics, and scoring—were rigorously validated to ensure smooth operation. Paddle control logic was tested to confirm accurate position updates when buttons were pressed, with constraints preventing paddles from moving outside the display matrix. Ball movement was validated across the 8x8 matrix, including correct speed adjustments and collision detection with paddles and screen edges.

The scoring system was tested to confirm that the ball reset correctly when a player missed a shot and that the game halted when a player reached 9 points, as indicated on the seven-segment display.

5.3.2 Timing and Latency Analysis

The system clock was fine-tuned to divide the **10 MHz frequency** into functional components for display multiplexing and gameplay updates. The display operated at 1 kHz, ensuring flicker-free visuals, while the ball and paddle updates at 5 Hz provided smooth and controlled gameplay. Input

latency was minimized, with paddle positions updating instantaneously upon button presses, ensuring a responsive and enjoyable user experience.

5.3.3 Resource Utilization and Reliability

The project efficiently utilized CPLD resources to implement the game logic, dot matrix control, and scoring. Registers and variables (`p1_pos`, `ball_x`, `ball_y`, `score_p1`, and `score_p2`) were managed within hardware limits, avoiding overuse of logic gates. Power efficiency was achieved by active multiplexing and limiting LED usage to one row or column at any given time.

The system's reliability was tested under continuous operation, with no observed glitches in ball movement, paddle control, or scoring. Boundary conditions, such as paddle edge behavior and ball resets, were also validated for consistent results.

5.3.4 Debugging and Optimization

Throughout development, issues such as paddle jitter, inconsistent ball reflections, and timing mismatches were resolved through systematic debugging. Optimizing the display multiplexing logic and refining the clock divisions ensured stable and accurate performance across all components. Ball trajectory adjustments based on paddle collision points further added precision and realism to gameplay.

5.4 Limitations of Tools (PO(e))

The CPLD's limited logic resources imposed constraints on the complexity of the game. Advanced features such as sound effects, animations, or additional game mechanics were not implemented due to hardware limitations. The dot matrix display's **8x8 resolution** restricted the visual detail, preventing smoother animations or graphical enhancements. Despite these constraints, the design successfully delivered functional and engaging gameplay with minimal resources.

5.5 Impact Assessment (PO(f))

5.5.1 Educational Value

This project demonstrates the practical application of digital logic design, signal synchronization, and hardware programming. By implementing game mechanics like paddle control, ball movement, and scoring through hardware, it bridges the gap between theoretical concepts and hands-on application. The project serves as an educational tool for students and hobbyists interested in digital electronics, offering a tangible example of sequential and combinational logic integration.

5.5.2 Technical Skill Development

The design process required a thorough understanding of digital systems, including clock division, multiplexing, and real-time updates. Debugging challenges such as ball trajectory inconsistencies and timing mismatches enhanced problem-solving and optimization skills. The project also strengthened skills in resource-constrained design, which is valuable for embedded and hardware engineers.

5.5.3 Future Extensions

The modular nature of the design supports enhancements, such as adding scoring animations, integrating sound effects, or implementing AI-controlled paddles. Increasing the game’s difficulty with faster ball speeds, reduced paddle sizes, or multiple balls could provide further opportunities for innovation and learning.

5.6 Sustainability Evaluation (PO(g))

The CPLD-based design is inherently energy-efficient compared to microcontroller or CPU-based implementations. The use of multiplexing reduces active LED power consumption, and minimal hardware components contribute to lower material usage and reduced environmental impact. Additionally, the project’s modular architecture allows the hardware to be repurposed for other logic-based applications, promoting resource reusability.

5.7 Ethical Considerations (PO(h))

The project is intended purely for educational and demonstration purposes. While inspired by the classic **Pong** game, the implementation on CPLD is unique and acknowledges its source of inspiration. By focusing on non-commercial and ethical use, the project encourages innovation, learning, and knowledge sharing within the electronics community.

6 Reflection on Individual and Teamwork (PO(i))

6.1 Individual Contribution of Each Member

The team consisted of four members, each contributing unique skills and expertise to the project:

Working Topic	Contributors (ID)
---------------	-------------------

Project Topic Selection		2006112 2006122 2006126 2006128
Algori thm	Game mechanics	2006126 2006112
	Ball & paddle control	2006126
	Dot matrix display	2006126 2006112 2006122
	Score display	2006128 2006112
	Clock division	2006122 2006128
Coding and debugging		2006126
CPLD implementation		2006126 2006112 2006122 2006128

6.2 Diversity Statement of Team

Our team’s composition for this project stood out due to its diversity in experiences, perspectives, and backgrounds, which fostered a dynamic and collaborative work environment. Each member brought unique skills and expertise, driving our collective efficiency and creativity. This diversity enabled us to analyze the problem thoroughly and encouraged adaptability in our approach, allowing us to pivot and refine ideas when faced with challenges. The variety of cultural insights and viewpoints enriched our discussions and strengthened decision-making, while our ability to collaborate seamlessly ensured that the best ideas surfaced. By recognizing the power of diversity and fostering an adaptable mindset, we effectively leveraged individual strengths to deliver innovative and practical solutions that met the project’s objectives.

6.3 Logbook of Project Implementation

Date	Milestone Achieved
03/10/2024	Project assigned
04/11/2024	Project update
09/11/2024	Project board change
14/12/2024	Final update

7. Communication to External Stakeholders

7.1 Executive Summary

The **Two Player Pong Game** is a digital implementation of the classic Pong game using **Verilog HDL** on a **CPLD**. The game is displayed on an **8x8 LED matrix** where two players control paddles to hit a moving ball. This project demonstrates the application of digital design principles, finite state machines, and real-time hardware control. The simplicity of the game makes it ideal for demonstrating interactive applications on CPLDs.

7.2 User Manual

Objective: Play a competitive two-player Pong game on an LED matrix.

Controls:

- **Player 1:**
 - UP: Switch 1
 - DOWN: Switch 2
- **Player 2:**
 - UP: Switch 3
 - DOWN: Switch 4

Steps to Play:

1. Power ON the system.
2. Use switches to move paddles up and down.
3. Hit the ball back to your opponent.

Winning: Prevent the ball from passing your paddle and for final win have to get

7.3 YouTube Link

<https://www.youtube.com/watch?v=KwgmoqzcpjU&feature=youtu.be>

7. Project Management and Cost Analysis (PO(k))

We implemented this project on the CPLD board. In this project of “Two player ping pong game using Verilog”, we didn’t use any other external hardware and software. So this project costs us nothing.

8. Future Work (PO(l))

For this project named “Two player ping pong game using Verilog”, here are some future work suggestions:

1. Enhanced Control Algorithms for Ball Movement

The control algorithms could be refined to improve the accuracy of the ball's movement, especially in terms of its speed and trajectory. Advanced algorithms like predictive control or adaptive control could be explored.

This would involve optimizing the existing system's response to external factors like changes in velocity or ball spin, making the movement smoother and more realistic. Implementing real-time adjustments could also allow for a more dynamic interaction.

2. Integration with Sensors for Real-Time Feedback

Adding sensors like infrared or ultrasonic sensors to detect the ball's position in real time could enable more precise control and feedback.

These sensors could help track the ball's location and adjust the motor's behavior accordingly, improving the overall performance. For example, detecting the ball's trajectory could allow for dynamic adjustments in the system to prevent misalignment or errors in positioning.

3. Wireless Communication for Remote Control

Introducing wireless control via Bluetooth or Wi-Fi could make the system more flexible, allowing users to control the movement of the ping-pong ball from a distance.

This could include a mobile app or a remote controller that sends signals to the CPLD board, offering a convenient and interactive way to operate the system. It could also allow multiple users to interact with the setup simultaneously.

4. Multi-Ball Simulation and Interaction

The system could be expanded to simulate multiple ping pong balls, allowing for a more complex and engaging interaction. By adding more balls to the setup, the system could simulate a real ping pong game, where multiple balls are tracked and manipulated. This would require adjustments to the system's logic to handle multiple objects, as well as an increase in processing power.

5. Improved User Interface (UI) with Real-Time Visualization

A graphical user interface (GUI) could be created to visualize the movement of the ball and provide feedback to users. This UI could display the ball's trajectory, speed, and position on a screen in real-time. Users could interact with the system through this interface, adjusting parameters like speed, direction, or even ball behavior, improving the user experience.

6. Optimization for Low-Power Operation

To improve the system's energy efficiency, optimization techniques could be explored to reduce power consumption, especially in a portable setup. This could involve using low-power CPLD boards or adjusting the frequency of control signals. Power-saving modes could be implemented for idle states, ensuring that the system operates efficiently without compromising performance.

7. Machine Learning for Predictive Ball Behavior

Implementing machine learning algorithms to predict the behavior of the ping pong ball based on its environment could enhance system performance.

By analyzing data from sensors and user input, the system could learn how to adjust its control algorithms to predict and correct the ball's movement more accurately, making it smarter and more adaptive.

These suggestions would allow for significant expansion and refinement of your current project, making it more interactive, dynamic, and advanced.

9. Reference:

1. <https://aamodkore.github.io/more/reports/ppreport.pdf>
2. <https://stackoverflow.com/questions/11798447/how-do-you-control-two-different-objects-separatelytwo-player-pong-game>
3. <https://nandland.com/project-10-pong/>
4. <https://community.stencyl.com/index.php?topic=27795.0>
5. <https://www.mecs-press.org/ijisa/ijisa-v15-n6/IJISA-V15-N6-3.pdf>
6. <https://discussions.unity.com/t/scripting-ai-for-2d-pong-game/529318>
7. <https://github.com/ambroseled/UCFK4-Pong>
8. https://www.researchgate.net/publication/332842902_IMPLEMENTATION_OF_DOUBLE_PLAYER_PONG_VIDEO_GAME_USING_VHDL_-_16-7220_DESD_Project